Self-Directed CS Roadmap: Durable Systems Understanding

Core filter: every stage below has been true since the 1970s and will still be true in 20 years. No frameworks, no language-of-the-month. The goal isn't "finish a syllabus" — it's reaching the point where you can look at a piece of code and predict *why* it will be slow, before running it.

Phase 1 — C and Memory (you're here)

Resource: Computer Systems: A Programmer's Perspective (CS:APP)

- Pointers as addresses, not "magic references"
- Stack vs heap: what actually lives where, and why
- How a function call works — stack frames, return addresses
- Manual memory management — malloc/free, and what happens when you get it wrong
- What a segfault means at the hardware level (illegal memory access, not "a bug")

Milestone project: write a small program in C with a deliberate memory bug (buffer overflow, use-after-free), then find and explain it without a debugger — just by reasoning about memory layout.

Phase 2 — How the CPU Executes Code

- Fetch-decode-execute cycle
- Registers, instructions
- Read (don't necessarily write) x86-64 assembly — enough to look at compiler output (`gcc -S`) and recognize what your C became
- Branching and why mispredicted branches cost real time

Milestone project: compile a simple C function, read the generated assembly, and narrate out loud what each instruction is doing.

Phase 3 — Memory Hierarchy and Caching

- L1/L2/L3 cache, RAM, disk — the actual speed gap between each
- Cache lines and locality of reference
- Why `array of structs` vs `struct of arrays` performs differently
- Why a linked list is cache-hostile compared to an array, even with identical Big-O

Milestone project: write two versions of the same task (e.g., summing values) — one cache-friendly, one cache-hostile — and benchmark the real time difference. Seeing the gap yourself is the point.

Phase 4 — Operating System Fundamentals

- Processes vs threads, context switching, scheduling
- File descriptors and syscalls — what actually happens when a program reads a file or opens a socket
- Concurrency: race conditions, locks, deadlocks

Milestone project: write a small multithreaded C program, deliberately introduce a race condition, observe the inconsistent output, then fix it with a lock.

Phase 5 — Networking (durable layer only)

- TCP/IP model
- Sockets — what a connection actually is at the OS level
- What happens, step by step, when a program opens a URL

Milestone project: write a bare-bones TCP client/server in C that sends a message back and forth.

Phase 6 — Data Structures, By Hand, In C

Not for Big-O memorization — for feeling the memory management underneath the

abstraction.

- Linked list
- Hash table
- Binary tree

Milestone project: implement all three from scratch, no libraries, and be able to explain what memory allocation happens on every insert/delete.

What “done” looks like

You can look at a piece of code — in any language — and reason about:

- Where its data lives (stack, heap, cache-friendly or not)
- What the OS is doing underneath it (syscalls, scheduling, memory paging)
- Why it’s fast or slow, before you profile it

That’s the actual target of a CS education, stripped of the parts that don’t age well. Everything else (frameworks, specific languages, cloud tooling) sits *on top* of this and is fast to pick up once this is solid.